

多媒體技術與應用

第 2 組專案報告

Project 6

組別： 2

班級、姓名與學號：

電資二 113820016 林稚蓁

電資二 113820032 曾靖諺

日期：2026.06.10

1. 實驗內容：

結合之前在 Project 4 時做過的 YOLO 模型技術，以及 Project 1 時做過的 OPENCV 機器學習技術。利用使用 YOLOV4-Tiny 訓練，目標是讓模型能辨識出 4 種不同的路牌（當心行人、道路封閉、停車再開、鐵路平交道），並實際利用 ONNX 與 TensorRT 部屬到 jetbot 上，使得 jetbot 在行駛過程中能準確辨識路牌，並因應路牌而做出反應；同時利用 canny 辨識車道後取影像中心點，當車輛偏移則修正，使得 jetbot 能夠直線行駛。

2. 實驗過程及結果：

預期實驗結果

預期利用 canny 進行邊緣偵測，能夠清楚辨識車道線，並沿兩線中心正常行駛，偏移後自動修正回中心，且 jetbot 能即時辨識四種路牌：stop（停車再開）、slow（當心行人）、railway（鐵路平交道）、road close（道路封閉）。

- slow（當心行人）：jetbot 減速，離開後恢復速度行駛
- stop（停車再開）：jetbot 停止 2 秒後繼續行駛
- railway（鐵路平交道）：jetbot 停止 5 秒後繼續行駛
- road close（道路封閉）：jetbot 完全停止，不超過路牌

實際上的結果

模型成功辨識 slow、stop、railway 與 road close 號誌，並且能夠在 jupyter notebook 上輸出正確的號誌，jetbot 能在偵測到路牌後執行對應動作（停車、減速、暫停後繼續），符合預期的結果。

走直線功能透過 canny 邊緣偵測與 hough 轉換計算車道中心點，在 160 ± 15 內視為直行區間，偏左或偏右時分別調整左右輪速差，整體行駛路徑大致直線。

遇到的問題&問題怎麼解決

(1) ModuleNotFoundError: No module named 'utils'

執行 Project06.ipynb 時，import utils 相關模組失敗，因為 python 工作目錄不在 trt_yolov4-tiny-master 資料夾內。

解決方法：利用 `os.chdir('trt_yolov4-tiny-master')` 切換到正確的目錄。

(2) jetbot 直線行駛過程向左偏移

jetbot 移動距離增加後，因為兩個車輪速度不一致，因此 jetbot 會向左偏移，導致衝出車道線。

解決方法：略為增加左輪速度，使得車輛能夠直線行駛。

(3) 路牌辨識與動作問題

偵測到道路封閉的標誌後，因為是在 while(1)這種無限迴圈內做判斷，之後使用 robot.stop()，因此只要突然有一幀沒偵測到就會衝出去；同理當心行人也是類似情形，只要有一幀沒偵測到就會恢復正常速度衝出去。停車再開與鐵路平交道則是可能會重複辨識而導致不會移動。

解決方法：改以全域變數 road_close 與 slow_mode 管理車輛移動狀態：若 road_close = 1 則不允許移動；slow_mode = 1 則車速降至原本的 0.5；stop 與 railway 標誌以 stop_ignore / railway_ignore 實現冷卻。

(4) 車道偵測失效

原始 canny 閾值設定 3000 / 2000，因為光線問題導致車道線並非純白色，幾乎沒有偵測到導致車道偵測失效。make_points()當斜率接近零時還是直接做除法，除以 0 導致座標整個異常，紅線顯示於畫面外。

解決方法：將 canny 閾值降低至 200 / 200，使車道線能夠被正確辨識出來，同時 average()加入條件 (abs(slope)小於 0.5 或大於 5 排除)，有效去除雜訊。make_points()補上斜率 = 0 的保護 (slope 小於 0.01 時強制設為 0.01) 及 x 座標夾值 (0~320)，避免數值爆炸。

(5) memory 不足

模型載入後 memory 使用率長期維持在 95%以上，而 getAverageLines()每幀將影像 resize 至 640x480，再執行 GaussianBlur(5,5)、Canny 與 HoughLinesP，運算量太大導致車輛反應遲鈍。

解決方法：將 getAverageLines()解析度縮小至 320x240，GaussianBlur kernel 從(5,5)縮為(3,3)。

(6) 引入 PD 控制器

因為單純的比例控制會導致過度補償，使 jetbot 在高速行駛的情況下會震盪與甩尾。

解決方法：在 modifySpeed 中實作 $d_error = error - prev_error$ 與 $correction = (error * 0.0004) + (d_error * 0.0002)$ ，引入微分項(d_error)預測誤差變化率，提供轉向阻尼，並將每一幀的基礎速度重置。

(7) 虛擬中線補償機制

當攝影機僅捕捉到單側車道線的時候，傳統求平均的方法會導致中線瞬間跳變至單側，造成 jetbot 暴衝。

解決方法：在 average 中加入了 virtual_mid[2] -= 30 與 += 30 的偏移邏輯，當失去一側車道線時，利用單側斜率投影出虛擬車道，並透過加上 30 像素的偏移量，維持穩定。

(8) 緊急停車機制

車輛在不小心偏離賽道或攝影機被遮住的時候，會重複上一個指令要做的動作，導致可能開始亂衝。

解決方法：update 與 modifySpeed 中實作了 if averaged_lines is None: robot.stop()，當偏離賽道或攝影機被遮住的時候自動停止 jetbot，避免亂衝。

3. 本次實驗過程說明與解決方法：

使用 jetbot 上的相機拍攝 4 種不同的車牌，並增加隨機排列組合後的影像，使用 roboflow 代替 labelImg 標記 YOLOV4-Tiny 模型需要的 bounding box。

上傳資料集至 colab，cfg 檔 batch=64、subdivisions=16、classes=4、filters=27、max_batches=8000 進行訓練。

訓練完成後將 weight 檔與 cfg 檔改名 yolov4-tiny-416.weights / yolov4-tiny-416.cfg，透過 yolo_to_onnx.py 轉換為 ONNX 格式，再透過 onnx_to_tensorrt.py 轉換為 TensorRT，部署到 jetbot 上。

利用 Project6.ipynb，整合 canny 邊緣偵測、Hough 車道偵測、差速修正演算法、偵測到路牌後的反應邏輯，使 jetbot 能即時辨識並依據路牌作出相對應的反應。

moudule not found 透過 os.chdir 正確指定目錄後解決；向左偏移問題透過增加左輪速度後解決；路牌偵測問題透過全域變數 road_close、slow_mode、stop_ignore、railway_ignore 解決；車道偵測失效問題透過降低 canny 閾值、增加斜率條件以及等於 0 的邊界問題判斷解決；memory 不足透過降低影像解析度以及降低高斯模糊的 kernal 解決。

4. 分工：

學號、組員	貢獻比例	工作內容
113820016 林稚蓁	30%	收集資料
113820032 曾靖諺	70%	訓練模型、Jetbot 路標偵測與走直線、小組報告